

MoveNet Pose Detection

Project untuk deteksi pose manusia menggunakan MoveNet model dari Google. Tersedia dalam 2 versi:

1. **Python Script** - untuk testing dan verifikasi model
2. **React Native Expo** - untuk mobile app development

Daftar Isi

- [Tentang Model](#)
- [Python Testing Script](#)
- [React Native Expo App](#)
- [Model Conversion](#)
- [Troubleshooting](#)



Tentang Model

Model: MoveNet SinglePose Lightning v4

Format: TensorFlow Lite (float16 quantized)

License: Apache 2.0

Source: Google TensorFlow

Spesifikasi Model

- **Input:** 192x192x3 RGB image (uint8, range 0-255)
- **Output:** [1, 1, 17, 3] tensor
 - 17 keypoints: nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles
 - 3 values per keypoint: [y, x, confidence]
 - Coordinates normalized to [0.0, 1.0]

17 Keypoints

1. nose
2. left_eye
3. right_eye
4. left_ear
5. right_ear
6. left_shoulder
7. right_shoulder
8. left_elbow
9. right_elbow
10. left_wrist
11. right_wrist
12. left_hip
13. right_hip
14. left_knee

15. right_knee
16. left_ankle
17. right_ankle

Python Testing Script

Prerequisites

```
# Install dependencies
pip install tensorflow
pip install opencv-python
pip install matplotlib
pip install numpy
pip install ai-edge-litert # atau tensorflow-lite
```

Cara Menggunakan

1. Pastikan model ada di folder yang sama:

```
project_izin/
├── 4.tflite
└── test_movenet.py
```

2. Jalankan script:

```
python test_movenet.py
```

3. Masukkan path ke gambar:

```
Masukkan path ke image yang ingin dideteksi:
Path: /path/to/your/image.jpg
```

4. Hasil akan ditampilkan:

- Console output dengan koordinat keypoints
- Visualisasi gambar dengan skeleton overlay
- File hasil: **result_pose_detection.jpg** dan **result_visualization.png**

Contoh Output

```
=====
DETECTED KEYPOINTS
=====
No      Keypoint          Y          X          Confidence
-----
1       nose             0.2345    0.5123    0.9456
2       left_eye          0.2134    0.4892    0.9234
...
=====
```

Class `MoveNetDetector`

```
from test_movenet import MoveNetDetector

# Initialize detector
detector = MoveNetDetector('4.tflite')

# Detect pose
keypoints, original_image = detector.detect('image.jpg')

# Print keypoints
detector.print_keypoints(keypoints)

# Draw visualization
result = detector.draw_keypoints(original_image, keypoints)
```

React Native Expo App

⚠ PENTING: Model Conversion

PROF

File `4.tflite` harus di-convert ke format TensorFlow.js terlebih dahulu!

TensorFlow Lite model tidak bisa langsung digunakan di React Native. Anda perlu:

Option 1: Convert TFLite ke TFJS

```
# Install tensorflowjs converter
pip install tensorflowjs

# Convert model
tensorflowjs_converter \
  --input_format=tf_saved_model \
  --output_format=tfjs_graph_model \
  --signature_name=serving_default \
  --saved_model_tags=serve \
```

```
/path/to/saved_model \
./react-native-movenet/assets/models
```

CATATAN: File `.tflite` perlu di-convert dari SavedModel format terlebih dahulu.

Option 2: Download Pre-converted Model

Download versi TFJS dari MoveNet:

```
# Download dari TensorFlow Hub
# https://tfhub.dev/google/tfjs-model/movenet/singlepose/lightning/4
```

Prerequisites

```
# Install Node.js dan npm
# https://nodejs.org/

# Install Expo CLI
npm install -g expo-cli
```

Setup Project

1. Masuk ke folder React Native:

```
cd react-native-movenet
```

2. Install dependencies:

```
npm install
```

3. Copy model ke assets:

```
mkdir -p assets/models
# Copy converted model files ke assets/models/
```

4. Update model path di `src/models/MoveNetModel.js`:

```
const modelJson = require('../assets/models/model.json');
const modelWeights = require('../assets/models/group1-
```

```
shard.bin');
```

Menjalankan App

```
# Start Metro bundler
npm start

# Atau langsung jalankan di platform tertentu:
npm run android # Untuk Android
npm run ios      # Untuk iOS (macOS only)
npm run web      # Untuk Web
```

Struktur Folder

```
react-native-movenet/
├─ App.js                # Main app component
├─ package.json
├─ app.json
├─ assets/
│  └─ models/            # Model files (converted to TFJS)
│     ├── model.json
│     └─ group1-shard.bin
└─ src/
   ├── components/
   │  └─ PoseOverlay.js  # Komponen untuk draw keypoints
   ├── models/
   │  ├── MoveNetModel.js # Model handler
   │  └─ PoseDetector.js  # High-level API
   └─ utils/
      ├── constants.js    # Keypoint names, edges, colors
      └─ imagePreprocessing.js # Image preprocessing utilities
```

PROF

Cara Menggunakan dalam Kode

```
import PoseDetector from './src/models/PoseDetector';

// Initialize
await PoseDetector.initialize();

// Detect dari tensor
const result = await PoseDetector.detectPose(imageTensor);

// Result structure:
{
  keypoints: [
```

```

{
  name: 'nose',
  index: 0,
  position: { x: 0.5, y: 0.3 },
  confidence: 0.95,
  visible: true
},
// ... 16 more keypoints
],
stats: {
  totalKeypoints: 17,
  visibleKeypoints: 15,
  averageConfidence: 0.87,
  detectionQuality: 'excellent'
},
timestamp: 1703345678901
}

```

Model Conversion (Detail)

Langkah-langkah Convert TFLite ke TFJS

Karena file `4.tflite` adalah quantized model, prosesnya lebih kompleks:

1. Download SavedModel Original:

```

# Download dari TensorFlow Hub
wget https://tfhub.dev/google/movenet/singlepose/lightning/4?tf-
hub-format=compressed

```

2. Convert SavedModel ke TFJS:

```

tensorflowjs_converter \
  --input_format=tf_saved_model \
  --output_node_names='output' \
  --saved_model_tags=serve \
  ./saved_model \
  ./tfjs_model

```

3. Atau gunakan model yang sudah di-convert:

- Visit: <https://tfhub.dev/google/tfjs-model/movenet/singlepose/lightning/4>
- Download dan extract ke `assets/models/`

Alternative: Menggunakan TFLite di React Native

Jika ingin tetap menggunakan `.tflite`:

1. Install react-native-tensorflow-lite:

```
npm install react-native-tensorflow-lite
```

2. Update kode untuk menggunakan TFLite interpreter

🔧 Troubleshooting

Python Script

Error: "No module named 'ai_edge_litert'"

```
pip install ai-edge-litert  
# atau  
pip install tensorflow-lite
```

Error: "Cannot read image"

- Pastikan path gambar benar
- Gunakan format: JPG, PNG, BMP
- Gunakan absolute path

Error: Model shape mismatch

- Pastikan menggunakan file `4.tflite` yang benar
- Model harus versi yang input uint8

React Native

PROF

Error: "Cannot find module 'model.json'"

- Pastikan model sudah di-convert ke TFJS format
- Copy files ke `assets/models/`
- Update require() path dengan benar

Error: "TensorFlow.js backend not initialized"

```
// Pastikan menunggu tf.ready()  
await tf.ready();
```

Performance Issues

- Gunakan model quantized (Float16)

- Reduce image resolution sebelum inference
- Implementasikan frame skipping untuk video

Memory Leaks

- Selalu dispose tensors setelah digunakan
- Gunakan `tf.tidy()` untuk auto cleanup



Development Roadmap

Fitur yang Sudah Tersedia

- ✓ Python testing script
- ✓ Image preprocessing utilities
- ✓ Keypoint detection
- ✓ Skeleton visualization
- ✓ React Native structure setup

Fitur yang Akan Dikembangkan

- ☐ Real-time camera detection
- ☐ Video file processing
- ☐ Multiple person detection
- ☐ Pose classification (yoga, exercise)
- ☐ Performance optimization
- ☐ Cloud deployment



License

Model: Apache 2.0 (Google TensorFlow)

Code: MIT License

PROF



Credits

- Model by Google TensorFlow Team
- MoveNet: <https://tfhub.dev/google/movenet>
- Documentation: <https://www.tensorflow.org/hub/tutorials/movenet>



Support

Untuk pertanyaan atau issues, silakan buka GitHub Issues atau hubungi developer.

Happy Coding! 🚀